

Constrained Minkowski Sum Selection and Finding

Cheng-Wei Luo, Peng-An Chen and Hsiao-Fei Liu

Department of Computer Science and Information Engineering
National Taiwan University, Taipei, Taiwan 106

Abstract

Let $P, Q \subseteq \mathbb{R}^2$ be two n -point multisets and $Ar \geq b$ be a set of λ inequalities on x and y , where $A \in \mathbb{R}^{\lambda \times 2}$, $r = \begin{bmatrix} x \\ y \end{bmatrix}$, and $b \in \mathbb{R}^\lambda$. Define the *constrained Minkowski sum* $(P \oplus Q)_{Ar \geq b}$ as the multiset $\{(p+q) | p \in P, q \in Q, A(p+q) \geq b\}$. Given $P, Q, Ar \geq b$, an objective function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, and a positive integer k , the MINKOWSKI SUM SELECTION PROBLEM is to find the k^{th} largest objective value among all objective values of points in $(P \oplus Q)_{Ar \geq b}$. Given $P, Q, Ar \geq b$, an objective function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, and a real number δ , the MINKOWSKI SUM FINDING PROBLEM is to find a point (x^*, y^*) in $(P \oplus Q)_{Ar \geq b}$ such that $|f(x^*, y^*) - \delta|$ is minimized. For the MINKOWSKI SUM SELECTION PROBLEM with linear objective functions, we obtain the following results: (1) optimal $O(n \log n)$ time algorithms for $\lambda = 1$; (2) $O(n \log^2 n)$ time algorithms for any fixed $\lambda > 1$. For the MINKOWSKI SUM FINDING PROBLEM with linear objective functions or objective functions of the form $f(x, y) = \frac{by}{ax}$, based on the algorithms proposed by Bernholt *et al.* (2007), we construct optimal $O(n \log n)$ time algorithms for any fixed $\lambda \geq 1$. As a byproduct, we obtain improved algorithms for the LENGTH-CONSTRAINED SUM SELECTION PROBLEM and the DENSITY FINDING PROBLEM.

1 Introduction

Let $P, Q \subseteq \mathbb{R}^2$ be two n -point multisets and $Ar \geq b$ be a set of λ inequalities on x and y , where $A \in \mathbb{R}^{\lambda \times 2}$, $r = \begin{bmatrix} x \\ y \end{bmatrix}$, and $b \in \mathbb{R}^\lambda$. Define the *constrained Minkowski sum* $(P \oplus Q)_{Ar \geq b}$ as the multiset

$$\{(p+q) | p \in P, q \in Q, A(p+q) \geq b\}.$$

In the MINKOWSKI SUM OPTIMIZATION PROBLEM, we are given $P, Q, Ar \geq b$, and an objective

function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$. The goal is to find the maximum objective value among all objective values of points in $(P \oplus Q)_{Ar \geq b}$. A function $f : D \subseteq \mathbb{R}^2 \rightarrow \mathbb{R}$ is said to be *quasiconvex* if and only if for all points $v_1, v_2 \in D$ and all $\gamma \in [0, 1]$, one has $f(\gamma \cdot v_1 + (1 - \gamma) \cdot v_2) \leq \max(f(v_1), f(v_2))$. Bernholt *et al.* [5] studied the MINKOWSKI SUM OPTIMIZATION PROBLEM for quasiconvex objective functions and showed that their results have applications to many sequence analysis problems arising in computational biology [1, 10, 11, 16, 17, 19]. In this paper, two variations of the MINKOWSKI SUM OPTIMIZATION PROBLEM are studied: the MINKOWSKI SUM SELECTION PROBLEM and the MINKOWSKI SUM FINDING PROBLEM.

In the MINKOWSKI SUM SELECTION PROBLEM, we are given $P, Q, Ar \geq b$, an objective function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, and a positive integer k . The goal is to find the k^{th} largest objective value among all objective values of points in $(P \oplus Q)_{Ar \geq b}$. The MINKOWSKI SUM OPTIMIZATION PROBLEM is equivalent to the MINKOWSKI SUM SELECTION PROBLEM with $k = 1$, and some known selection problems, like the SUM SELECTION PROBLEM [3, 14], the LENGTH-CONSTRAINED SUM SELECTION PROBLEM [15], and the SLOPE SELECTION PROBLEM [6, 7, 8, 12, 18], are reducible to the MINKOWSKI SUM SELECTION PROBLEM in linear time.

In the MINKOWSKI SUM FINDING PROBLEM, we are given $P, Q, Ar \geq b$, an objective function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, and a real number δ . The goal is to find a point (x^*, y^*) in $(P \oplus Q)_{Ar \geq b}$ such that $|f(x^*, y^*) - \delta|$ is minimized. We shall show that the DENSITY FINDING PROBLEM recently proposed by Lee *et al.* [13] is reducible to the MINKOWSKI SUM FINDING PROBLEM in linear time.

Our main results are as follows.

- The MINKOWSKI SUM SELECTION PROBLEM with one constraint and a linear objective function can be solved in optimal $O(n \log n)$ time.

- The MINKOWSKI SUM SELECTION PROBLEM with two constraints and a linear objective function can be solved in $O(n \log^2 n)$ time.
- For any fixed $\lambda > 2$, the MINKOWSKI SUM SELECTION PROBLEM with λ constraints and a linear objective function is shown to be asymptotically equivalent to the MINKOWSKI SUM SELECTION PROBLEM with two constraints and a linear objective function.
- The MINKOWSKI SUM FINDING PROBLEM with any fixed number of constraints can be solved in optimal $O(n \log n)$ time if the objective function $f(x, y)$ is linear or of the form $\frac{by}{ax}$.

As a byproduct, we obtain improved algorithms for the LENGTH-CONSTRAINED SUM SELECTION PROBLEM [15] and the DENSITY FINDING PROBLEM [13]. Recently, Lin and Lee [15] proposed an expected $O(n \log(u - l + 1))$ -time randomized algorithm for the LENGTH-CONSTRAINED SUM SELECTION PROBLEM, where n is the size of the input instance and $l, u \in \mathbb{N}$ are two given parameters with $1 \leq l < u \leq n$. Lee, Lin, and Lu [13] showed the DENSITY FINDING PROBLEM has a lower bound of $\Omega(n \log n)$ and proposed an $O(n \log^2 m)$ -time algorithm for it, where n is the size of the input instance and m is a parameter whose value may be as large as n . In this paper, we obtain an $O(n \log(u - l + 1))$ -time deterministic algorithm for the LENGTH-CONSTRAINED SUM SELECTION PROBLEM and an optimal $O(n \log n)$ -time algorithm for the DENSITY FINDING PROBLEM.

2 Preliminaries

In the following, we review some definitions and theorems. For more details, readers can refer to [5, 9]. A matrix $X \in \mathbb{R}^{n \times m}$ is said to be *sorted* if the values of each row and each column are in nondecreasing order. Frederickson and Johnson [9] gave some results about the selection problem and the ranking problem in a collection of sorted matrices. From the results of Frederickson and Johnson [9], we have the following theorems.

Theorem 1: The selection problem in a collection of sorted matrices is given a rank k and a collection of sorted matrices $\{X_1, \dots, X_N\}$ in which X_j has dimensions $n_j \times m_j$, $n_j \geq m_j$, to find the k^{th} largest element among all elements of sorted matrices in $\{X_1, \dots, X_N\}$. This problem can be solved in $O(\sum_{j=1}^N m_j \log(2n_j/m_j))$ time.

Theorem 2: The ranking problem in a collection of sorted matrices is given an element and a collection of sorted matrices $\{X_1, \dots, X_N\}$ in which X_j has dimensions $n_j \times m_j$, $n_j \geq m_j$, to find the rank of the given element among all elements of sorted matrices in $\{X_1, \dots, X_N\}$. This problem can be solved in $O(\sum_{j=1}^N m_j \log(2n_j/m_j))$ time.

By the recent works of Bernholt *et al.* [5], we have the following theorems.

Theorem 3: Given a set of λ linear inequalities $Ar \geq b$ and two n -point multisets $P, Q \subseteq \mathbb{R}^2$, one can compute a set $R \subseteq (P \oplus Q)_{Ar \geq b}$ containing all vertices of the convex hull of $(P \oplus Q)_{Ar \geq b}$ in $O(\lambda \log \lambda + \lambda \cdot n \log n)$ time. The number of vertices in R is bounded by $O(\lambda \cdot n \log n)$.

Theorem 4: The problem of maximizing a quasiconvex objective function f over the constrained Minkowski sum $(P \oplus Q)_{Ar \geq b}$ requires $\Omega(n \log n)$ time in the algebraic decision tree model even if f is a linear function and $Ar \geq b$ consists of only one constraint.

3 Minkowski Sum Selection with One Constraint

In this section we study the MINKOWSKI SUM SELECTION PROBLEM and give an optimal $O(n \log n)$ time algorithm for the case where only one linear constraint is given and the objective function is also linear.

3.1 Input Transformation

Given $P = \{(x_{1,1}, y_{1,1}), \dots, (x_{1,n}, y_{1,n})\}$, $Q = \{(x_{2,1}, y_{2,1}), \dots, (x_{2,n}, y_{2,n})\}$, a positive integer k , one constraint $L: ax + by \geq c$, and a linear objective function $f(x, y) = dx + ey$, where a, b, c, d , and e are all constants, we perform the following transformation.

1. Change the content of P and Q to $\{(ax_{1,1} + by_{1,1}, dx_{1,1} + ey_{1,1}), \dots, (ax_{1,n} + by_{1,n}, dx_{1,n} + ey_{1,n})\}$, and $\{(ax_{2,1} + by_{2,1}, dx_{2,1} + ey_{2,1}), \dots, (ax_{2,n} + by_{2,n}, dx_{2,n} + ey_{2,n})\}$, respectively.
2. Change the constraint from $ax + by \geq c$ to $x \geq c$.
3. Change the objective function from $dx + ey$ to y .

This transformation can be done in $O(n)$ time and the answer remains the same. Hence from now on, our goal becomes to find the k^{th} largest y -coordinate on the constrained Minkowski sum of P and Q subject to the constraint $L: x \geq c$.

3.2 Algorithm

For ease of exposition, we assume that no two points in P and Q have the same x -coordinate and n is a power of two. The algorithm proceeds as follows. First, we sort P and Q into P_x and Q_x (P_y and Q_y , respectively) in nondecreasing order of x -coordinates (y -coordinates, respectively) in $O(n \log n)$ time. Next, we use a divide-and-conquer approach to store the y -coordinates of $(P \oplus Q)_{x \geq c}$ as a collection of sorted matrices and then apply Theorem 1 to selecting the k^{th} largest element from the elements of these sorted matrices.

We now explain how to store the y -coordinates of $(P \oplus Q)_{x \geq c}$ as a collection of sorted matrices. Let $P_x = ((x_1, y_1), \dots, (x_n, y_n))$, $Q_x = ((\bar{x}_1, \bar{y}_1), \dots, (\bar{x}_n, \bar{y}_n))$, $P_y = ((x'_1, y'_1), \dots, (x'_n, y'_n))$, and $Q_y = ((\bar{x}'_1, \bar{y}'_1), \dots, (\bar{x}'_n, \bar{y}'_n))$. First of all, we divide P_x into two halves of equal size: $A = ((x_1, y_1), \dots, (x_{n/2}, y_{n/2}))$ and $B = ((x_{n/2+1}, y_{n/2+1}), \dots, (x_n, y_n))$. We then find a point $(\bar{x}_t, \bar{y}_t)$ of Q_x such that $x_{n/2} + \bar{x}_t < c$ and t is maximized. Divide Q_x into two halves: $C = ((\bar{x}_1, \bar{y}_1), \dots, (\bar{x}_t, \bar{y}_t))$ and $D = ((\bar{x}_{t+1}, \bar{y}_{t+1}), \dots, (\bar{x}_n, \bar{y}_n))$. $(P \oplus Q)_{x \geq c}$ is the union of $(A \oplus C)_{x \geq c}$, $(A \oplus D)_{x \geq c}$, $(B \oplus C)_{x \geq c}$, and $(B \oplus D)_{x \geq c}$. Because $\bar{x}_t$ is the largest x -coordinate among all x -coordinates of points in Q_x such that $x_{n/2} + \bar{x}_t < c$, we know that the points in $A \oplus C$ cannot satisfy the constraint $x \geq c$. Hence, we only need to consider points in $A \oplus D$, $B \oplus C$, and $B \oplus D$. Because P_x and Q_x are in nondecreasing order of x -coordinates, it is guaranteed that all points in $B \oplus D$ satisfy the constraint L , i.e., $B \oplus D = (B \oplus D)_{x \geq c}$. Construct in linear time $row_vector = (r_1, r_2, \dots, r_{n/2})$ which is the y -coordinates in the subsequence of P_y resulting from removing points with x -coordinates no greater than $x_{n/2}$ from P_y . Construct in linear time $column_vector = (c_1, c_2, \dots, c_{n-t})$ which is the y -coordinates in the subsequence of Q_y resulting from removing points with x -coordinates no greater than $\bar{x}_t$ from Q_y . Note row_vector is the same as the result of sorting B into non-decreasing order of y -coordinates, and $column_vector$ is the same as the result of sorting D into non-decreasing

order of y -coordinates. Thus, we have $\{y : (x, y) \in (B \oplus D)_{x \geq c}\} = \{y : (x, y) \in B \oplus D\} = \{r_i + c_j : 1 \leq i \leq |row_vector|, 1 \leq j \leq |column_vector|\}$. Therefore, we can store the y -coordinates of $B \oplus D = (B \oplus D)_{x \geq c}$ as a sorted matrix X of dimensions $|row_vector| \times |column_vector|$ where the (i, j) -th element of X is $r_i + c_j$. Note that it is not necessary to explicitly construct the sorted matrix X , which needs $\Omega(n^2)$ time. Because the (i, j) -th element of X can be obtained by summing up r_i and c_j , we only need to keep row_vector and $column_vector$. The rest is to construct the sorted matrices for the y -coordinates of points in $(A \oplus D)_{x \geq c}$ and $(B \oplus C)_{x \geq c}$. It is accomplished by applying the above approach recursively.

The pseudocode is shown in Figure 1 and Figure 2. We now analyze the time complexity.

Lemma 1: Given a matrix $X \in \mathbb{R}^{\mathfrak{N} \times \mathfrak{M}}$, we define the *side length* of X be $\mathfrak{N} + \mathfrak{M}$. Letting $T(n', m')$ be the time complexity of running $\text{ConstructMatrices}(P_x, Q_x, P_y, Q_y, L)$, where $n' = |P_x| = |P_y|$ and $m' = |Q_x| = |Q_y|$, we have $T(n', m') = O((n' + m') \log(n' + 1))$. Similarly, letting $M(n', m')$ be the sum of the side lengths of all sorted matrices created by running $\text{ConstructMatrices}(P_x, Q_x, P_y, Q_y, L)$, we have $M(n', m') = O((n' + m') \log(n' + 1))$.

Proof: It suffices to prove that $T(n', m') = O((n' + m') \log(n' + 1))$. By Algorithm ConstructMatrices in Figure 1, we have

$$T(n', m') \leq \max_{0 \leq i \leq m'} \{c'(n' + m') + T(n'/2, i) + T(n'/2, m' - i)\}, \text{ for some constant } c'.$$

Then by induction on n' , it is easy to prove that $T(n', m')$ is $O((n' + m') \log(n' + 1))$. $\square$

Theorem 5: Given two n -point multisets $P \subseteq \mathbb{R}^2$ and $Q \subseteq \mathbb{R}^2$, a positive integer k , a linear constraint L , and a linear objective function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, Algorithm Selection_1 finds the k^{th} largest objective value among all objective values of points in $(P \oplus Q)_L$ in $O(n \log n)$ time. Hence, by Theorem 4 we obtain Algorithm Selection_1 is optimal.

Proof: Let $S = \{X_1, \dots, X_N\}$ be the sorted matrices produced at Step 4 in Algorithm Selection_1 . Let X_j , $1 \leq j \leq N$, be of dimensions $n_j \times m_j$ where $n_j \geq m_j$. By Lemma 1, we have $O(\sum_{i=1}^N (m_i + n_i)) = O(n \log n)$. By Theorem 1,

Algorithm ConstructMatrices($P_x, Q_x, P_y, Q_y, L: x \geq c$)

Input: P_x and P_y are the results of sorting the multiset $P \subseteq \mathbb{R}^2$ in nondecreasing order of x -coordinates and y -coordinates, respectively. Q_x and Q_y are the results of sorting the multiset $Q \subseteq \mathbb{R}^2$ in nondecreasing order of x -coordinates and y -coordinates, respectively. A linear constraint $L: x \geq c$.

Output: Store the y -coordinates of points in $(P \oplus Q)_{x \geq c}$ as a collection of sorted matrices.

```

1   $n' \leftarrow |P_x|$ ;  $m' \leftarrow |Q_x|$ ;
2  Let  $P_x = ((x_1, y_1), \dots, (x_{n'}, y_{n'}))$ ,  $Q_x = ((\bar{x}_1, \bar{y}_1), \dots, (\bar{x}_{m'}, \bar{y}_{m'}))$ ,
    $P_y = ((x'_1, y'_1), \dots, (x'_{n'}, y'_{n'}))$ , and  $Q_y = ((\bar{x}'_1, \bar{y}'_1), \dots, (\bar{x}'_{m'}, \bar{y}'_{m'}))$ .
3   $\bar{x}_0 \leftarrow -\infty$ ;
4  if  $n' \leq 0$  or  $m' \leq 0$  then
5      return;
6  if  $n' = 1$  or  $m' = 1$  then
7      Scan points in  $P_x \oplus Q_x$  to find all points satisfying  $L$  and construct the sorted matrix for
        $y$ -coordinates of these points.
8      return;
9  for  $t \leftarrow m'$  downto 0 do
10     if  $x_{n'/2} + \bar{x}_t < c$  then
11          $row\_vector \leftarrow$  subsequence of  $P_y$  resulting from removing points with  $x$ -coordinates  $\leq x_{n'/2}$ ;
12          $column\_vector \leftarrow$  subsequence of  $Q_y$  resulting from removing points with  $x$ -coordinates  $\leq \bar{x}_t$ ;
13          $A_x \leftarrow P_x[1, n'/2]$ ;  $B_x \leftarrow P_x[n'/2 + 1, n']$ ;  $C_x \leftarrow Q_x[1, t]$ ;  $D_x \leftarrow Q_x[t + 1, m']$ ;
14          $A_y \leftarrow$  subsequence of  $P_y$  resulting from removing points with  $x$ -coordinates  $> x_{n'/2}$ ;
15          $B_y \leftarrow$  subsequence of  $P_y$  resulting from removing points with  $x$ -coordinates  $\leq x_{n'/2}$ ;
16          $C_y \leftarrow$  subsequence of  $Q_y$  resulting from removing points with  $x$ -coordinates  $> \bar{x}_t$ ;
17          $D_y \leftarrow$  subsequence of  $Q_y$  resulting from removing points with  $x$ -coordinates  $\leq \bar{x}_t$ ;
18         ConstructMatrices( $A_x, D_x, A_y, D_y, L: x \geq c$ );
19         ConstructMatrices( $B_x, C_x, B_y, C_y, L: x \geq c$ );
20     return;
```

Figure 1: The subroutine for the MINKOWSKI SUM SELECTION PROBLEM with one constraint.

Algorithm Selection₁(P, Q, L, f, k)

Input: Two multisets $P \subseteq \mathbb{R}^2$ and $Q \subseteq \mathbb{R}^2$; a linear constraint L ; a linear objective function $f: \mathbb{R}^2 \rightarrow \mathbb{R}$; a positive integer k .

Output: The k^{th} largest objective value among all objective values of points in $(P \oplus Q)_L$.

```

1  Perform the input transformation described in Section 3.1.
2  Sort  $P$  and  $Q$  into  $P_x$  and  $Q_x$ , respectively, in nondecreasing order of their  $x$ -coordinates.
3  Sort  $P$  and  $Q$  into  $P_y$  and  $Q_y$ , respectively, in nondecreasing order of their  $y$ -coordinates.
4   $S \leftarrow$  ConstructMatrices( $P_x, Q_x, P_y, Q_y, L$ );
5  Return the  $k^{th}$  largest element among the elements of sorted matrices in  $S$ .
```

Figure 2: The main procedure for the MINKOWSKI SUM SELECTION PROBLEM with one constraint.

the time required to find the k^{th} largest element from the elements of matrices in S is $O(\sum_{i=1}^N m_i \log(2n_i/m_i))$. It follows that

$$\begin{aligned} \sum_{i=1}^N m_i \log(2n_i/m_i) &\leq \sum_{i=1}^N m_i \frac{2n_i}{m_i} \\ &= \sum_{i=1}^N 2n_i \leq \sum_{i=1}^N 2(m_i + n_i). \end{aligned}$$

Thus, we obtain the time bound $O(n \log n)$. Combining this time with the input transformation time, the sorting time, and the execution time of `ConstructMatrices`(P_x, Q_x, P_y, Q_y, L), we conclude that the total running time is $O(n \log n)$. $\square$

Using similar techniques, the following problem can also be solved in $O(n \log n)$ time. Given two n -point multisets $P \subseteq \mathbb{R}^2$ and $Q \subseteq \mathbb{R}^2$, a linear constraint L , a linear objective function $f: \mathbb{R}^2 \rightarrow \mathbb{R}$, and a real number t , the problem is to find the rank of t among all objective values of points in $(P \oplus Q)_L$, where the rank of t is equal to the number of elements in $\{y | (x, y) \in (P \oplus Q)_L, y > t\}$ plus one. The pseudocode is given in Figure 3. Note that in Algorithm `Ranking1` and Algorithm `Selection1`, we assume the input linear constraint L is of the form $ax + by \geq c$. We only slightly modify the pseudocode in Algorithm `ConstructMatrices`, and then we can solve the input linear constraint L of the form $ax + by > c$. For the redundancy of the pseudocode, we omit the details here. We shall assume Algorithm `Ranking1` and Algorithm `Selection1` are capable of solving problems with the linear constraint L of the forms $ax + by \geq c$ and $ax + by > c$ in the following section.

4 Minkowski Sum Selection with Multiple Constraints

Let λ be a fixed integer greater than one. In this section, we shall prove that the MINKOWSKI SUM SELECTION PROBLEM can be solved in $O(n \log^2 n)$ time for the case where λ linear constraints are given and the objective function is linear.

4.1 Minkowski Sum Selection with Two Constraints

In the following we show the MINKOWSKI SUM SELECTION PROBLEM can be solved in worst-case

$O(n \log^2 n)$ time for the case where two linear constraints are given and the objective function is linear.

4.1.1 Input Transformation

Given $P = \{(x_{1,1}, y_{1,1}), \dots, (x_{1,n}, y_{1,n})\}$, $Q = \{(x_{2,1}, y_{2,1}), \dots, (x_{2,n}, y_{2,n})\}$, a positive integer k , two constraints $L_1: a_1x + b_1y \geq c_1$ and $L_2: a_2x + b_2y \geq c_2$, and a linear objective function $f(x, y) = dx + ey$, where $a_1, b_1, c_1, a_2, b_2, c_2, d$, and e are all constants, we perform the following transformation.

1. Change the content of P and Q to $\{(a_1x_{1,1} + b_1y_{1,1}, dx_{1,1} + ey_{1,1}), \dots, (a_1x_{1,n} + b_1y_{1,n}, dx_{1,n} + ey_{1,n})\}$, and $\{(a_1x_{2,1} + b_1y_{2,1}, dx_{2,1} + ey_{2,1}), \dots, (a_1x_{2,n} + b_1y_{2,n}, dx_{2,n} + ey_{2,n})\}$, respectively.
2. Change the constraints from $a_1x + b_1y \geq c_1$ and $a_2x + b_2y \geq c_2$ to $x \geq c_1$ and $\frac{a_2e - b_2d}{a_1e - b_1d}x + \frac{a_1b_2 - b_1a_2}{a_1e - b_1d}y \geq c_2$, respectively.
3. Change the objective function from $dx + ey$ to y .

This transformation can be done in $O(n)$ time and the answer remains the same. Hence from now on, our goal becomes to find the k^{th} largest y -coordinate on the constrained Minkowski sum of P and Q subject to the constraints $L_1: x \geq c_1$ and $L_2: ax + by \geq c_2$, where $a = \frac{a_2e - b_2d}{a_1e - b_1d}$ and $b = \frac{a_1b_2 - b_1a_2}{a_1e - b_1d}$. Note that if the two constraints and the objective function are parallel, we cannot use the above transformation. However, if the two constraints are parallel, this problem can be solved in $O(n \log n)$ time. The algorithm for this special case is similar to our improved algorithm for the LENGTH-CONSTRAINED SUM SELECTION PROBLEM described later. Hence, we omit the details here.

4.1.2 Algorithm

After applying the above input transformation to our problem instances, there will be four possible cases: (1) $a < 0, b < 0$; (2) $a > 0, b > 0$; (3) $a < 0, b > 0$; (4) $a > 0, b < 0$. See Figure 4 for an illustration. Note that the two constraints are not parallel implies both $a \neq 0$ and $b \neq 0$. In the following discussion we shall focus on Case (1), and the other three cases can be solved in a similar way.

Algorithm $\text{Ranking}_1(P, Q, L, f, t)$

Input: Two multisets $P \subseteq \mathbb{R}^2$ and $Q \subseteq \mathbb{R}^2$; a linear constraint L ; a linear objective function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$; a real number t .

Output: The rank of t among the objective values of points in $(P \oplus Q)_L$.

- 1 Perform the input transformation in Section 3.1.
- 2 Sort P and Q into P_x and Q_x , respectively, in nondecreasing order of their x -coordinates.
- 3 Sort P and Q into P_y and Q_y , respectively, in nondecreasing order of their y -coordinates.
- 4 $S \leftarrow \text{ConstructMatrices}(P_x, Q_x, P_y, Q_y, L)$;
- 5 Return the rank of t among the elements of sorted matrices in S .

Figure 3: The ranking subroutine for the Minkowski sum of P and Q with one constraint.

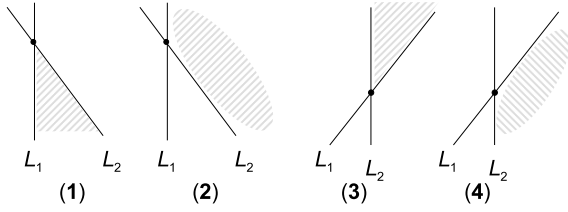


Figure 4: An illustration of four possible cases after the input transformation.

For simplicity, we assume that n is a power of two. The pseudocode is shown in Figure 5 and Figure 6. First we sort P and Q into P_x and Q_x (P_y and Q_y , respectively) in nondecreasing order of x -coordinates (y -coordinates, respectively) using $O(n \log n)$ time. Let $P_x = \{(x_1, y_1), \dots, (x_n, y_n)\}$, $Q_x = \{(\bar{x}_1, \bar{y}_1), \dots, (\bar{x}_n, \bar{y}_n)\}$, $P_y = \{(x'_1, y'_1), \dots, (x'_n, y'_n)\}$, and $Q_y = \{(\bar{x}'_1, \bar{y}'_1), \dots, (\bar{x}'_n, \bar{y}'_n)\}$. Denote by Y the sorted matrix of dimensions $n \times n$ where the (i, j) -th element is $y'_i + \bar{y}'_j$. Since Y is composed of all y -coordinates of points in $P \oplus Q$, the solution y^* is also in Y . In Algorithm Selection₂ shown in Figure 6, we shall run a while loop to find the solution y^* from the elements of Y . In each execution of the loop, we maintain an integer interval $[l, u]$ such that $y^* \in \{\text{the } h^{\text{th}} \text{ largest element of } Y : l \leq h \leq u\}$. Initially, we set $l = 1$ and $u = n^2$. At the beginning of each iteration, we select the $\frac{u-l+1}{2}$ -th largest element t of Y . The $\frac{u-l+1}{2}$ -th largest element t of Y can be found in $O(n)$ time by Theorem 1. Let $U = (\alpha, \beta)$ be the intersection point of the line $x = c_1$ and the line $ax + by = c_2$. Then there are four possible cases: (i) $\beta < t$; (ii) $y^* < t \leq \beta$; (iii) $t = y^*$; (iv) $t < y^*$. See Figure 7 for an illustration. If it is Case (i) or Case (ii), then we reset l to $\frac{u-l+1}{2}$ and continue to the next iteration. If it is Case (iii), then we stop the loop

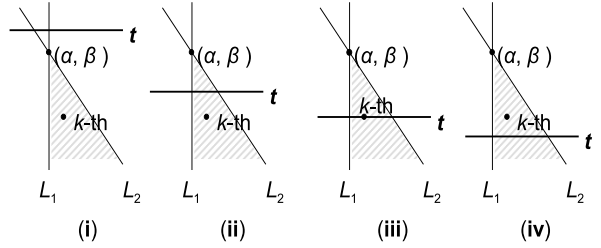


Figure 7: The four possible cases for a given value t .

and return t . If it is Case (iv), then we reset u to $\frac{u-l+1}{2}$ and continue to the next iteration. In the following we explain how to distinguish these four cases.

If $t > \beta$, then we know it is Case (i). Otherwise, we shall do the following to identify that it is Case (ii), Case (iii), or Case (iv). Let $A = \{(x, y) | x < c_1 \text{ and } ax + by < c_2\}$, $B = \{(x, y) | x < c_1 \text{ and } ax + by \geq c_2 \text{ and } y > t\}$, $C = \{(x, y) | x \geq c_1 \text{ and } ax + by \geq c_2 \text{ and } y > t\}$ and $D = \{(x, y) | x \geq c_1 \text{ and } ax + by < c_2 \text{ and } y > t\}$. See Figure 8 for an illustration. Let R be the rank of t among the values of y -coordinates of the points in region C , i.e., $R = |\{y | (x, y) \in (P \oplus Q)_{L_1, L_2} \text{ and } y > t\}| + 1$. If $R < k$ then it is Case (ii). If $R = k$ then it is Case (iii). If $R > k$ then it is Case (iv). It remains to explain how to compute R . First, we compute the number of points in $(P \oplus Q) \cap (A \cup B)$, say $R_1 = \text{Ranking}_1(P, Q, L': x < c_1, f'(x, y) = y, t) - 1$, and the number of points in $(P \oplus Q) \cap (A \cup D)$, say $R_2 = \text{Ranking}_1(P, Q, L'': ax + by < c_2, f'(x, y) = y, t) - 1$. Then we compute the number of points in $(P \oplus Q) \cap A$, say $R_3 = \text{Ranking}_1(P, Q, L''': ax + by < c_2, f''(x, y) = -x, c_1) - 1$. Finally, we must compute the number of points in $(P \oplus Q)_{y > t}$, say R_t . It can be done by re-calculating the rank

Algorithm Ranking₂(P, Q, L_1, L_2, f, t)

Input: Two multisets $P \subseteq \mathbb{R}^2$ and $Q \subseteq \mathbb{R}^2$; two linear constraints L_1 and L_2 ;
a linear objective function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$; a real number t .

Output: The rank of t among the objective values of points in $(P \oplus Q)_{L_1, L_2}$.

- 1 Perform the input transformation in Section 4.1.1. Let $L_1: x \geq c_1$ and $L_2: ax + by \geq c_2$ after the input transformation.
- 2 Sort P and Q into P_x and Q_x , respectively, in nondecreasing order of their x -coordinates.
- 3 Sort P and Q into P_y and Q_y , respectively, in nondecreasing order of their y -coordinates.
- 4 Let $P_x = ((x_1, y_1), \dots, (x_n, y_n))$, $Q_x = ((\bar{x}_1, \bar{y}_1), \dots, (\bar{x}_n, \bar{y}_n))$, $P_y = ((x'_1, y'_1), \dots, (x'_n, y'_n))$, and $Q_y = ((\bar{x}'_1, \bar{y}'_1), \dots, (\bar{x}'_n, \bar{y}'_n))$.
- 5 $R_1 \leftarrow \text{Ranking}_1(P, Q, L': x < c_1, f'(x, y) = y, t) - 1$;
- 6 $R_2 \leftarrow \text{Ranking}_1(P, Q, L'': ax + by < c_2, f'(x, y) = y, t) - 1$;
- 7 $R_3 \leftarrow \text{Ranking}_1(P, Q, L'': ax + by < c_2, f''(x, y) = -x, c_1) - 1$;
- 8 Denote by Y the sorted matrix of dimensions $n \times n$ where the (i, j) -th element is $y'_i + \bar{y}'_j$.
- 9 $R_t \leftarrow$ the rank of t among the values of y -coordinates of the points in Y minus one;
- 10 $R \leftarrow R_t - R_1 - R_2 + R_3 + 1$;
- 11 **return** R ;

Figure 5: The ranking subroutine for the MINKOWSKI SUM SELECTION PROBLEM with two constraints.

Algorithm Selection₂(P, Q, L_1, L_2, f, k)

Input: $P \subseteq \mathbb{R}^2$ and $Q \subseteq \mathbb{R}^2$ are two multisets; L_1 and L_2 are linear constraints; $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ is a linear objective function; k is a positive integer.

Output: The k^{th} largest objective value among all objective values of points in $(P \oplus Q)_{L_1, L_2}$.

- 1 Perform the input transformation in Section 4.1.1. Let $L_1: x \geq c_1$ and $L_2: ax + by \geq c_2$ after the transformation.
- 2 Sort P and Q into P_x and Q_x , respectively, in nondecreasing order of the x -coordinates.
- 3 Sort P and Q into P_y and Q_y , respectively, in nondecreasing order of the y -coordinates.
- 4 Let $P_x = ((x_1, y_1), \dots, (x_n, y_n))$, $Q_x = ((\bar{x}_1, \bar{y}_1), \dots, (\bar{x}_n, \bar{y}_n))$, $P_y = ((x'_1, y'_1), \dots, (x'_n, y'_n))$, and $Q_y = ((\bar{x}'_1, \bar{y}'_1), \dots, (\bar{x}'_n, \bar{y}'_n))$.
- 5 Denote by Y the sorted matrix of dimensions $n \times n$ where the (i, j) -th element is $y'_i + \bar{y}'_j$.
- 6 Calculate the intersection point $U = (\alpha, \beta)$ of L_1 and L_2 .
- 7 $u \leftarrow n^2$; $l \leftarrow 1$; $m \leftarrow \frac{u-l+1}{2}$; $t \leftarrow$ the m^{th} largest element of Y ;
- 8 **while** true **do**
- 9 **if** $t > \beta$ **then**
- 10 $l \leftarrow m$; $m \leftarrow \frac{u+l}{2}$; $t \leftarrow$ the m^{th} largest element of Y ;
- 11 **else**
- 12 $R \leftarrow \text{Ranking}_2(P, Q, L_1, L_2, f, t)$;
- 13 **if** $R < k$ **then**
- 14 $l \leftarrow m$; $m \leftarrow \frac{u+l}{2}$; $t \leftarrow$ the m^{th} largest element of Y .
- 15 **else if** $R = k$ **then**
- 16 **return** t ;
- 17 **else**
- 18 $u \leftarrow m$; $m \leftarrow \frac{u-l+1}{2}$; $t \leftarrow$ the m^{th} largest element of Y .

Figure 6: The main procedure for the MINKOWSKI SUM SELECTION PROBLEM with two constraints.

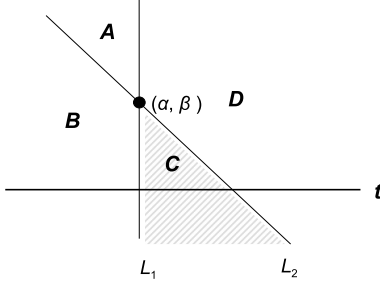


Figure 8: The four regions above (exclusive) the line $y = t$.

of t among the values of the elements in Y , say R'_t , and R_t is equal to $R'_t - 1$. By Theorem 2, we can solve it easily. Note that R'_t is not equal to the original rank of t among the values of the elements in Y if the y -coordinate of the value t is not unique, i.e., there are several points having the same value of y -coordinate, t . After getting R_1, R_2, R_3 , and R_t , we can compute R by the following equation:

$$R = R_t - R_1 - R_2 + R_3 + 1.$$

Now we analyze the time complexity. Since the **while**-loop in Algorithm Selection₂ consists of at most $O(\log n)$ iterations and each iteration takes $O(n \log n)$ time resulted from the execution of Algorithm Ranking₂, the total complexity of the loop is $O(n \log^2 n)$. By combining this time with the input transformation time, sorting time, we have the following theorem.

Theorem 6: Given two n -point multisets $P \subseteq \mathbb{R}^2$ and $Q \subseteq \mathbb{R}^2$, a positive integer k , two linear constraints L_1 and L_2 , and a linear objective function $f: \mathbb{R}^2 \rightarrow \mathbb{R}$, Algorithm Selection₂ can find the k^{th} largest objective value among all objective values of points in $(P \oplus Q)_{L_1, L_2}$ in $O(n \log^2 n)$ time.

Note that the pseudocode of Algorithm Selection₂ in Figure 6 is constructed only for Case (1), i.e., $a < 0$ and $b < 0$ for a given constraint $L_2: ax + by \geq c_2$, but we can slightly modify the pseudocode to obtain those for the remaining three cases. For the ease of the redundant pseudocodes, we shall only use Selection₂ to represent the pseudocode for all cases in the following section.

4.2 Minkowski Sum Selection with $\lambda > 2$ Constraints

In the following, we shall explain how to solve the MINKOWSKI SUM SELECTION with λ constraints using Algorithm Selection₂ and Algorithm

Ranking₂ in $O(\lambda \cdot n \log n + \lambda \log \lambda + n \log^2 n)$ time, where $\lambda > 2$. Applying the similar input transformation stated in Section 4.1.1, we can assume the objective function is $f(x, y) = y$ without loss of generality. Let $\chi = \{L_1, L_2, \dots, L_\lambda\}$ be the set of λ constraints and $C = \{(x, y) | (x, y) \in (P \oplus Q)_{L_1, L_2, \dots, L_\lambda}\}$. Let $V = (v_1 = (x_1, y_1), v_2 = (x_1, y_2), \dots, v_m = (x_m, y_m))$ be the vertices of the polygon formed by the λ constraints, sorted in nonincreasing order of y -coordinates. All points in C are inside this polygon. Computing V can be done in $O(\lambda \log \lambda)$ time [4]. Let $\theta = \{(L_1^i, L_2^i) | L_1^i \text{ and } L_2^i \text{ are the two constraints in this polygon between the lines } y = y_i \text{ and } y = y_{i+1}, i = 1 \text{ to } m-1\}$. The set θ can be easily constructed in $O(\lambda \log \lambda)$ time when we compute V . For $i = 1$ to $m-1$, we define the region $R_i = \{(x, y) | (x, y) \in C \text{ satisfying the constraints } y \leq y_i \text{ and } y > y_{i+1}\}$. See Figure 9 for an illustration.

In the following, we explain the details of Algorithm Selection _{λ} . The pseudocode is shown in Figure 10. First of all, we compute the value $|R_i|$, where $i = 1, 2, \dots, m-1$. $|R_i|$ is equal to the number of points in $(P \oplus Q)_{L_1^i, L_2^i}$ above the line $y = y_{i+1}$ minus the number of points in $(P \oplus Q)_{L_1^i, L_2^i}$ above the line $y = y_i$. The number of points in $(P \oplus Q)_{L_1^i, L_2^i}$ above the line $y = y_{i+1}$ can be obtained by calculating the rank of y_{i+1} among the values of y -coordinates of the points in $(P \oplus Q)_{L_1^i, L_2^i}$ minus one. We can calculate this rank by calling Algorithm Ranking₂($P, Q, L_1^i, L_2^i, f(x, y) = y, y_{i+1}$). We can also obtain the number of points in $(P \oplus Q)_{L_1^i, L_2^i}$ above the line $y = y_i$ in the same way. Next, we compute $S_i = |R_1| + |R_2| + \dots + |R_i|$. The value S_i means the number of points in $(P \oplus Q)_{L_1, L_2, \dots, L_\lambda}$ above the line $y = y_{i+1}$. Then we run a for loop to find the region R_i where the solution y^* is contained. If $S_{w-1} < k < S_w$, we know that y^* is in the region R_w . In order to use Algorithm Selection₂, we call Ranking₂($P, Q, L_1^w, L_2^w, f(x, y) = y, y_{w-1}$) to find the rank of y_{w-1} among the values of y -coordinates of points in $(P \oplus Q)_{L_1^w, L_2^w}$. Let this rank be r . As a result, we can obtain the rank of y^* among the y -coordinates of points in $(P \oplus Q)_{L_1^w, L_2^w}$ is equal to $r + k - S_{w-1} - 1$. Finally, we call Selection₂($P, Q, L_1^w, L_2^w, f(x, y) = y, r + k - S_{w-1} - 1$) to get the solution y^* .

Because there are λ constraints, it takes $O(\lambda \log \lambda)$ time to compute V and θ . The execution time of Algorithm Selection _{λ} is dominated by the first for loop and the invocation of Selection₂. This for loop call Algorithm Ranking₂ at most λ

Algorithm $\text{Selection}_\lambda(P, Q, \chi, V, \theta, k)$

Input: Two multisets $P \subseteq \mathbb{R}^2$ and $Q \subseteq \mathbb{R}^2$; a set χ of the constraints $L_1, \dots, L_\lambda$; a set V of all vertices of the polygon formed by the λ constraints; a set θ as defined in Section 4.2; a positive integer k .

Output: The k^{th} largest value of y -coordinates among all points in $(P \oplus Q)_{L_1, \dots, L_\lambda}$.

```

1   $m \leftarrow |V|$ ;  $S_0 \leftarrow 0$ ;
2  for  $i \leftarrow 1$  to  $m - 1$  do
3       $R_i \leftarrow \text{Ranking}_2(P, Q, L_1^i, L_2^i, f(x, y) = y, y_{i+1}) - \text{Ranking}_2(P, Q, L_1^i, L_2^i, f(x, y) = y, y_i)$ ;
4       $S_i \leftarrow 0$ ;  $S_i \leftarrow R_i + S_{i-1}$ ;
5  if  $k = 1$  then
6      return  $y_1$ ;
7  if  $k = n^2$  then
8      return  $y_m$ ;
9  for  $i \leftarrow 1$  to  $m - 1$  do
10     if  $S_i = k$  then
11         return  $y_i$ ;
12     else if  $S_i > k$  then
13          $w \leftarrow i - 1$ ;
14         break;
15  $r \leftarrow \text{Ranking}_2(P, Q, L_1^w, L_2^w, f(x, y) = y, y_{w-1})$ ;
16 return  $\text{Selection}_2(P, Q, r + k - S_{w-1} - 1, L_1^w, L_2^w, f(x, y) = y)$ ;

```

Figure 10: The pseudocode for the MINKOWSKI SUM SELECTION PROBLEM with λ constraints.

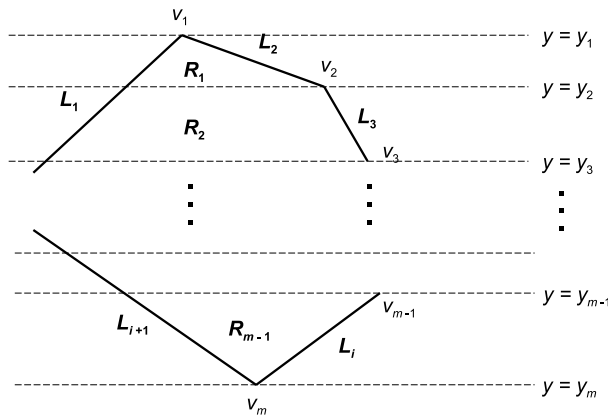


Figure 9: An illustration of the polygon formed by λ constraints.

times, taking $O(\lambda \cdot n \log n)$. The invocation of Selection_2 takes $O(n \log^2 n)$ time by Theorem 6. Hence, we obtain that the total time complexity is $O(\lambda \cdot n \log n + \lambda \log \lambda + n \log^2 n)$. We conclude the time complexity in the following theorem.

Theorem 7: Let λ be any fixed integer larger than two. The MINKOWSKI SUM SELECTION PROBLEM with λ constraints and a linear objective function is asymptotically equivalent to the MINKOWSKI SUM SELECTION PROBLEM with two linear constraints and a linear objective function.

5 Minkowski Sum Finding with Multiple Constraints

In the MINKOWSKI SUM FINDING PROBLEM, given two n -point multisets P, Q , a set of λ inequalities $Ax \geq b$, an objective function $f(x, y)$ and a real number δ , we are required to find a point $v^* = (x^*, y^*)$ among all points in $(P \oplus Q)_{Ax \geq b}$ which minimizes $|f(x, y) - \delta|$.

5.1 Objective Function $f(x, y) = ax + by$

We change the content of P to $\{(x_{1,1}, y_{1,1} - \frac{\delta}{2b}), (x_{2,1}, y_{2,1} - \frac{\delta}{2b}), \dots, (x_{n,1}, y_{n,1} - \frac{\delta}{2b})\}$ and the content of Q to $\{(x_{1,2}, y_{1,2} - \frac{\delta}{2b}), (x_{2,2}, y_{2,2} - \frac{\delta}{2b}) -$

$\frac{\delta}{2b}), \dots, (x_{n,2}, y_{n,2} - \frac{\delta}{2b})\}$. We also change the inequalities $L_i: a_i x + b_i y \geq c_i$ to $a_i x + b_i y \geq c_i - \frac{b_i}{b} \delta$, $i = 1, 2, \dots, \lambda$. The objective function is changed to $ax + by + \delta$. Thus, we turn to find a point $v_1^* = (x_1^*, y_1^*)$ among all points in $(P \oplus Q)_{Ar \geq b}$ which minimizes $|ax_1^* + by_1^*|$.

We divide the xy -plane into two parts: $ax + by \geq 0$ and $ax + by < 0$. Then we can derive the following lemma.

Lemma 2: Given two points $v_1 = (x_1, y_1)$, $v_2 = (x_2, y_2)$, both satisfy $ax + by \geq 0$ or $ax + by < 0$, let $v_\gamma = (x_\gamma, y_\gamma) = \gamma v_1 + (1 - \gamma)v_2$, where $\gamma \in [0, 1]$. Then we have $|ax_\gamma + by_\gamma| \geq \min(|ax_1 + by_1|, |ax_2 + by_2|)$.

Proof: It is easy to see the lemma holds if $b = 0$. Without loss of generality, let $x_1 \geq x_2$ and $b \neq 0$. It suffices to prove the case v_1, v_2 both satisfy $ax + by \geq 0$; the other case can be proved in a similar way. We consider the following two cases: (1) $|ax_1 + by_1| \leq |ax_2 + by_2|$; (2) $|ax_1 + by_1| > |ax_2 + by_2|$. In the first case, by $|ax_1 + by_1| \leq |ax_2 + by_2|$, $ax_1 + by_1 \geq 0$ and $ax_2 + by_2 \geq 0$, we can derive $b(y_2 - y_1) \geq a(x_1 - x_2)$. Let $v' = (x', y')$ satisfy $ax_1 + by_1 = ax' + by'$ and $x' = x_\gamma = \gamma x_1 + (1 - \gamma)x_2$; so $y' = \frac{a}{b}(1 - \gamma)(x_1 - x_2) + y_1$. Since $y_\gamma = \gamma y_1 + (1 - \gamma)y_2 = (1 - \gamma)(y_2 - y_1) + y_1$, $by_\gamma \geq by'$. Thus, $|ax_\gamma + by_\gamma| \geq |ax' + by'| = |ax_1 + by_1|$. In the second case, $b(y_1 - y_2) > a(x_2 - x_1)$. Let $v'' = (x'', y'')$ satisfy $ax_2 + by_2 = ax'' + by''$ and $x'' = x_\gamma = \gamma x_1 + (1 - \gamma)x_2$; so $y'' = \frac{a}{b}\gamma(x_2 - x_1) + y_2$. Since $y_\gamma = \gamma y_1 + (1 - \gamma)y_2 = \gamma(y_1 - y_2) + y_2$, $by_\gamma > by''$. Thus, $|ax_\gamma + by_\gamma| > |ax'' + by''| = |ax_2 + by_2|$. Therefore, $|ax_\gamma + by_\gamma| \geq \min(|ax_1 + by_1|, |ax_2 + by_2|)$ if $ax_1 + by_1 \geq 0$ and $ax_2 + by_2 \geq 0$. $\square$

By Lemma 2, the points in $(P \oplus Q)_{Ar \geq b}$ which satisfy $ax + by \geq 0$ (or $ax + by < 0$) and minimize $|ax + by|$ are on the vertices of the convex hull of all points in $(P \oplus Q)_{Ar \geq b}$ which satisfy $ax + by \geq 0$ (or $ax + by < 0$). By Theorem 3, we can compute two point sets R_1 and R_2 : R_1 contains all vertices of the convex hull of all points in $(P \oplus Q)_{Ar \geq b}$ which satisfy $ax + by \geq 0$; R_2 contains all vertices of the convex hull of all points in $(P \oplus Q)_{Ar \geq b}$ which satisfy $ax + by < 0$. The number of vertices in R_1 and R_2 is bounded by $O(\lambda \cdot n \log n)$. Thus, the point $v_1^* = (x_1^*, y_1^*)$ can be found by examining all points in R_1 and R_2 and we can obtain the solution $v^* = (x^*, y^*) = (x_1^*, y_1^* + \frac{\delta}{b})$.

5.2 Objective Function $f(x, y) = \frac{by}{ax}$

Without loss of generality, we assume the x -coordinates of all points in $(P \oplus Q)_{Ar \geq b}$ are nonzero. First, we change the content of P to $\{(ax_{1,1}, -a\delta x_{1,1} + by_{1,1}), (ax_{2,1}, -a\delta x_{2,1} + by_{2,1}), \dots, (ax_{n,1}, -a\delta x_{n,1} + by_{n,1})\}$ and the content of Q to $\{(ax_{1,2}, -a\delta x_{1,2} + by_{1,2}), (ax_{2,2}, -a\delta x_{2,2} + by_{2,2}), \dots, (ax_{n,2}, -a\delta x_{n,2} + by_{n,2})\}$ subject to $a, b \neq 0$. We also change the inequalities $L_i: a_i x + b_i y \geq c_i$ to $(\frac{a_i}{a} + \frac{\delta b_i}{b})x + \frac{b_i}{b}y \geq c_i$, $i = 1, 2, \dots, \lambda$. The objective function is changed to $\frac{y}{x} + \delta$. After the transformation, the answer of this problem is not changed. Thus, we turn to find a point $v_2^* = (x_2^*, y_2^*)$ among all points in $(P \oplus Q)_{Ar \geq b}$ which minimizes $|\frac{y_2^*}{x_2^*}|$.

Similar to Section 5.1, let $D_1 = \{(x, y) | x > 0, y \geq 0\}$, $D_2 = \{(x, y) | x < 0, y \geq 0\}$, $D_3 = \{(x, y) | x < 0, y \leq 0\}$ and $D_4 = \{(x, y) | x > 0, y \leq 0\}$. In each D_i , we have the following lemma.

Lemma 3: Let $D_1 = \{(x, y) | x > 0, y \geq 0\}$, $D_2 = \{(x, y) | x < 0, y \geq 0\}$, $D_3 = \{(x, y) | x < 0, y \leq 0\}$ and $D_4 = \{(x, y) | x > 0, y \leq 0\}$. Given two points $v_1 = (x_1, y_1)$, $v_2 = (x_2, y_2)$, v_1 and v_2 belong to the same D_i , $i = 1, 2, 3, 4$, let $v_\gamma = (x_\gamma, y_\gamma) = \gamma v_1 + (1 - \gamma)v_2$, where $\gamma \in [0, 1]$. Then we have $|\frac{y_\gamma}{x_\gamma}| \geq \min(|\frac{y_1}{x_1}|, |\frac{y_2}{x_2}|)$.

Proof: Without loss of generality, let $x_1 \geq x_2$. It suffices to prove the case where v_1, v_2 belong to D_1 ; the other cases can be proved in a similar way. Assume v_1, v_2 belong to D_1 . We consider the following two cases: (1) $|\frac{y_1}{x_1}| \leq |\frac{y_2}{x_2}|$; (2) $|\frac{y_1}{x_1}| > |\frac{y_2}{x_2}|$. In the first case, let $v' = (x', y')$ satisfy $\frac{y_1}{x_1} = \frac{y'}{x'}$ and $x' = x_\gamma = \gamma x_1 + (1 - \gamma)x_2$; so $y' = (\gamma x_1 + (1 - \gamma)x_2) \frac{y_1}{x_1} = \gamma y_1 + (1 - \gamma) \frac{x_2}{x_1} y_1$. Since v_1, v_2 are in D_1 , $|\frac{y_1}{x_1}| \leq |\frac{y_2}{x_2}|$ implies $y_2 \geq \frac{x_2}{x_1} y_1$. It follows that $y' \leq y_\gamma = \gamma y_1 + (1 - \gamma)y_2$, so $\frac{y_\gamma}{x_\gamma} \geq \frac{y'}{x'} = \frac{y_1}{x_1}$ implies $|\frac{y_\gamma}{x_\gamma}| \geq |\frac{y_1}{x_1}|$. In the second case, let $v'' = (x'', y'')$ satisfy $\frac{y_2}{x_2} = \frac{y''}{x''}$ and $x'' = x_\gamma = \gamma x_1 + (1 - \gamma)x_2$; so $y'' = (\gamma x_1 + (1 - \gamma)x_2) \frac{y_2}{x_2} = \gamma \frac{x_1}{x_2} y_2 + (1 - \gamma)y_2$. Since v_1, v_2 are in D_1 , $|\frac{y_1}{x_1}| > |\frac{y_2}{x_2}|$ implies $y_1 > \frac{x_1}{x_2} y_2$. It follows that $y'' < y_\gamma = \gamma y_1 + (1 - \gamma)y_2$, $\frac{y_\gamma}{x_\gamma} > \frac{y''}{x''} = \frac{y_2}{x_2}$ implies $|\frac{y_\gamma}{x_\gamma}| > |\frac{y_2}{x_2}|$. Therefore, we have $|\frac{y_\gamma}{x_\gamma}| \geq \min(|\frac{y_1}{x_1}|, |\frac{y_2}{x_2}|)$. $\square$

By Lemma 3, the points in $(P \oplus Q)_{Ar \geq b} \cap D_i$ which minimize $|\frac{y}{x}|$ are on the vertices of the convex hull of all points in $(P \oplus Q)_{Ar \geq b} \cap D_i$. By Theorem 3, we can compute four point sets

R_i , $i = \{1, 2, 3, 4\}$ and each of them contains all vertices of the convex hull of all points in $(P \oplus Q)_{Ar > b} \cap D_i$ and the number of vertices in R_i is bounded by $O(\lambda \cdot n \log n)$. Therefore, the point $v_2^* = (x_2^*, y_2^*)$ can be found by examining all points in R_i and we can obtain the solution $v^* = (x^*, y^*) = (\frac{x_2^*}{a}, \frac{x_2^*}{b}\delta + \frac{y_2^*}{b})$.

5.3 Lower Bound

Lemma 4 : The MINKOWSKI SUM FINDING PROBLEM with the objective function $f(x, y) = ax + by$ or $f(x, y) = \frac{by}{ax}$ has a lower bound of $\Omega(n \log n)$.

Proof: We will reduce the SET DISJOINTNESS PROBLEM to the MINKOWSKI SUM FINDING PROBLEM with the objective function $f(x, y) = ax + by$ or $f(x, y) = \frac{by}{ax}$. Given two sets $A = \{a_1, \dots, a_n\}$ and $B = \{b_1, \dots, b_n\}$, the SET DISJOINTNESS PROBLEM is to determine whether or not $A \cap B = \emptyset$. The SET DISJOINTNESS PROBLEM has a lower bound of $\Omega(n \log n)$ [2].

Given two sets $A = \{a_1, \dots, a_n\}$ and $B = \{b_1, \dots, b_n\}$, A, B are the inputs of the SET DISJOINTNESS PROBLEM. Let $P = \{(1, a_1), (1, a_2), \dots, (1, a_n)\}$, $Q = \{(1, -b_1), (1, -b_2), \dots, (1, -b_n)\}$ and $\delta = 0$ be the inputs for the MINKOWSKI SUM FINDING PROBLEM. The answer of the MINKOWSKI SUM FINDING PROBLEM with the objective function $f(x, y) = y$ equals to $(2, 0)$ if and only if $A \cap B \neq \emptyset$. Thus, the SET DISJOINTNESS PROBLEM is reduced to find the answer of the MINKOWSKI SUM FINDING PROBLEM with the objective function $f(x, y) = ax + by$. So the MINKOWSKI SUM FINDING PROBLEM with a linear objective function has a lower bound of $\Omega(n \log n)$. We can also reduce the SET DISJOINTNESS PROBLEM to the MINKOWSKI SUM FINDING PROBLEM with the objective function $f(x, y) = \frac{by}{ax}$ in a similar way, so the MINKOWSKI SUM FINDING PROBLEM with the objective function which is of the form $\frac{by}{ax}$ also has a lower bound of $\Omega(n \log n)$. $\square$

5.4 Time Analysis

The transformation of both two algorithms can be done in $O(n)$ time. Then we compute R_i , where $i = 1, 2, \dots, k$ ($k = 2$ for objective function $f(x, y) = ax + by$ and $k = 4$ for objective function $f(x, y) = \frac{by}{ax}$). The computation time of all R_i is bounded by $O(k\lambda \log \lambda + k\lambda n \log n)$. Lastly

we examine all points in the R_i ; this step cost $O(k\lambda n \log n)$ time. Since λ and k are constants, the total time is bounded by $O(n + k\lambda \log \lambda + k\lambda n \log n + k\lambda n \log n) = O(n \log n)$. By Lemma 4, we know this two problems have a lower bound of $\Omega(n \log n)$, so our algorithm is optimal. The results are summarized in the following theorem.

Theorem 8 : The MINKOWSKI SUM FINDING PROBLEM with the objective function $f(x, y) = ax + by$ or $f(x, y) = \frac{by}{ax}$ can be done in optimal $O(n \log n)$ time.

6 Applications

6.1 Length-constrained Sum Selection Problem

Given a sequence $S = (s_1, s_2, \dots, s_n)$ of n real numbers, and two positive integers l, u with $l < u$, define the length and sum of a segment $S[i, j] = (s_i, \dots, s_j)$ to be $length(i, j) = j - i + 1$ and $sum(i, j) = \sum_{h=i}^j s_h$, respectively. A segment is said to be feasible if and only if its length is in $[l, u]$. The LENGTH-CONSTRAINED SUM SELECTION PROBLEM is to find the k^{th} largest sum among all sums of feasible segments of S .

When there are no length constraints, i.e., $l = 1$ and $u = n$, the LENGTH-CONSTRAINED SUM SELECTION PROBLEM becomes the SUM SELECTION PROBLEM. Bengtsson and Chen [3] first studied the SUM SELECTION PROBLEM and gave an $O(n \log^2 n)$ -time algorithm for it. Recently, Lin and Lee provided an $O(n \log n)$ -time algorithm [14] for the SUM SELECTION PROBLEM and an expected $O(n \log(u - l + 1))$ -time randomized algorithm [15] for the LENGTH-CONSTRAINED SUM SELECTION PROBLEM. In the following, we show how to solve the LENGTH-CONSTRAINED SUM SELECTION PROBLEM in worst-case $O(n \log(u - l + 1))$ time.

6.1.1 Algorithm

We first reduce the LENGTH-CONSTRAINED SUM SELECTION PROBLEM to the MINKOWSKI SUM SELECTION PROBLEM as follows. Let $P = \{p_0, p_1, \dots, p_n\}$ and $Q = \{q_0, q_{-1}, \dots, q_{-n}\}$, where $p_i = (x_i, y_i) = (i, \sum_{t=1}^i s_t)$ and $q_{-i} = (\bar{x}_i, \bar{y}_i) = (-i, -\sum_{t=1}^i s_t)$ for all $i = 0, 1, \dots, n$. A point (x, y) in $P \oplus Q$ is said to be a feasible point if and only if $l \leq x \leq u$. Each feasible segment $S[i, j]$ corresponds to a feasible point $(x, y) = p_j + q_{1-i}$

in $P \oplus Q$. Thus, the LENGTH-CONSTRAINED SUM SELECTION PROBLEM is equivalent to finding the k^{th} largest y -coordinate among all y -coordinates of feasible points in $P \oplus Q$. We next show how to do this in $O(n \log(u-l+1))$ time. For simplicity, we assume n is a multiple of $u-l$.

1. Let $i_t = t(u-l)$ and $j_t = l - t(u-l)$ for $t = 0, 1, \dots, \frac{n}{u-l}$.
2. For $t \leftarrow 0$ to $\frac{n}{u-l}$ do
 - (a) Let $P_t = \{p_h \in P : i_t - (u-l) < h \leq i_t\}$ and $Q_t = Q_{t,1} \cup Q_{t,2}$, where $Q_{t,1} = \{q_h \in Q : j_t \leq h < j_t + (u-l)\}$ and $Q_{t,2} = \{q_h \in Q : j_t + (u-l) \leq h < j_t + 2(u-l)\}$.
 - (b) Store the y -coordinates of points in $(P_t \oplus Q_{t,1})_{x \geq l}$ as a set $N_{t,1}$ of sorted matrices such that the sum of side lengths of the sorted matrices in $N_{t,1}$ is no greater than $c \cdot (|P_t| + |Q_{t,1}|) \log(|P_t| + |Q_{t,1}| + 1)$ for some constant c .
 - (c) Store the y -coordinates of points in $(P_t \oplus Q_{t,2})_{x \leq u}$ as a set $N_{t,2}$ of sorted matrices such that the sum of side lengths of the sorted matrices in $N_{t,2}$ is no greater than $c \cdot (|P_t| + |Q_{t,2}|) \log(|P_t| + |Q_{t,2}| + 1)$ for some constant c .
3. Return the k^{th} largest element among the elements of sorted matrices in $\bigcup_{t=0}^{\frac{n}{u-l}} (N_{t,1} \cup N_{t,2})$.

The following lemma ensures the correctness.

Lemma 5 : $(P \oplus Q)_{l \leq x \leq u} = \bigcup_{t=0}^{\frac{n}{u-l}} ((P_t \oplus Q_{t,1})_{l \leq x} \cup (P_t \oplus Q_{t,2})_{x \leq u})$.

Proof: We shall prove that $(P \oplus Q)_{l \leq x \leq u} \stackrel{(1)}{=} \bigcup_{t=0}^{\frac{n}{u-l}} (P_t \oplus Q)_{l \leq x \leq u} \stackrel{(2)}{=} \bigcup_{t=0}^{\frac{n}{u-l}} (P_t \oplus Q_t)_{l \leq x \leq u} \stackrel{(3)}{=} \bigcup_{t=0}^{\frac{n}{u-l}} ((P_t \oplus Q_{t,1})_{l \leq x \leq u} \cup (P_t \oplus Q_{t,2})_{l \leq x \leq u}) \stackrel{(4)}{=} \bigcup_{t=0}^{\frac{n}{u-l}} ((P_t \oplus Q_{t,1})_{l \leq x} \cup (P_t \oplus Q_{t,2})_{x \leq u})$. It is clear that equations (1) and (3) are true, so only equations (2) and (4) remain to be proved.

We first prove equation (2) by showing that $(P_t \oplus Q)_{l \leq x \leq u} = (P_t \oplus Q_t)_{l \leq x \leq u}$. Suppose for contradiction that there exist $p_i \in P_t$ and $q_j \notin Q_t$ such that $l \leq x_i + \bar{x}_j = i + j \leq u$. By $p_i \in P_t$, we have $(t-1)(u-l) < i \leq t(u-l)$; by $q_j \notin Q_t$, we have either $j < l - t(u-l)$ or $j \geq l - (t-2)(u-l)$. It follows that $i + j$ is either less than l or larger than u , a contradiction.

To prove equation (4), it suffices to prove that all points in $(P_t \oplus Q_{t,1})$ must have x -coordinates less than u and all points in $(P_t \oplus Q_{t,2})$ must have x -coordinates larger than l . Let $p_i \in P_t$, $q_j \in Q_{t,1}$ and $q_{j'} \in Q_{t,2}$. It follows that $(t-1)(u-l) < x_i = i \leq t(u-l)$, $l - t(u-l) \leq \bar{x}_j = j < l - (t-1)(u-l)$, and $l - (t-1)(u-l) \leq \bar{x}_{j'} = j' < l - (t-2)(u-l)$. Thus, we have $x_i + \bar{x}_j = i + j < u$ and $l < x_i + \bar{x}_{j'} = i + j'$. $\square$

Since $|P_t|$, $|Q_{t,1}|$, and $|Q_{t,2}|$ are no greater than $(u-l)$ for all t , each execution of Step 2.b and Step 2.c can be done in $O((u-l) \log(u-l+1))$ time by Lemma 1. There are total $\frac{n}{u-l} + 1$ iterations of the **for**-loop in Step 2, so the total time spent on Step 2.b and Step 2.c is $O(n \log(u-l+1))$. The sum of side lengths of sorted matrices in $\bigcup_{t=0}^{\frac{n}{u-l}} (N_{t,1} \cup N_{t,2})$ is $O(\sum_{t=0}^{\frac{n}{u-l}} \sum_{i=1}^2 (|P_t| + |Q_{t,i}|) \log(|P_t| + |Q_{t,i}| + 1)) = O(\sum_{t=0}^{\frac{n}{u-l}} (u-l) \log(u-l+1)) = O(n \log(u-l+1))$. Therefore, by Theorem 1, Step 3 can be done in $O(n \log(u-l+1))$ time. Putting everything together, we have that the total running time is $O(n \log(u-l+1))$. The next theorem summarizes the results of this subsection.

Theorem 9 : The LENGTH-CONSTRAINED SUM SELECTION PROBLEM can be solved in $O(n \log(u-l+1))$ time.

6.2 Density Finding Problem

Given a sequence of number pairs $S = ((s_1, w_1), (s_2, w_2), \dots, (s_n, w_n))$ where $w_i > 0$ for $i = 1, 2, \dots, n$, two positive numbers l, u with $l \leq u$, and a real number δ , let *segment* $S(i, j)$ of S be the consecutive subsequence of S between indices i and j . Define the *sum* $s(i, j)$, *width* $w(i, j)$, and *density* $d(i, j)$ of segment $S(i, j)$ to be $\sum_{r=i}^j s_r$, $\sum_{r=i}^j w_r$ and $\frac{s(i, j)}{w(i, j)}$, respectively. A segment $S(i, j)$ is said to be *feasible* if and only if $l \leq w(i, j) \leq u$. The DENSITY FINDING PROBLEM is to find a feasible segment $S(i^*, j^*) = ((s_{i^*}, w_{i^*}), (s_{i^*+1}, w_{i^*+1}), \dots, (s_{j^*}, w_{j^*}))$ such that $|d(i^*, j^*) - \delta|$ is minimized. Lee *et al.* [13] proved that the DENSITY FINDING PROBLEM has a lower bound of $\Omega(n \log n)$ in the algebraic decision tree model and provided an $O(n \log^2 m)$ algorithm for it, where $m = \min(\lfloor \frac{u-l}{w_{\min}} \rfloor, n)$ and $w_{\min} = \min_{1 \leq r \leq n} w_r$. In the following we describe how to solve the DENSITY FINDING PROBLEM problem in $O(n \log n)$ time by using the algorithm developed in Section 5.2.

Let $w(1, 0) = 0$ and $s(1, 0) = 0$. First we construct in $O(n)$ time the following two point sets: $P = \{(w(1, i), s(1, i)) | 0 \leq i \leq n\}$ and $Q = \{(-w(1, i), -s(1, i)) | 0 \leq i \leq n\}$. Each feasible segment $S(i, j)$ of S corresponds to a point $(w(1, j) - w(1, i - 1), s(1, j) - s(1, i - 1))$ in $(P \oplus Q)_{l \leq x \leq u}$, where i, j are recorded when we compute every new points in $(P \oplus Q)_{l \leq x \leq u}$. Thus, the problem is equivalent to finding the point (x^*, y^*) in $(P \oplus Q)_{l \leq x \leq u}$ such that $|\frac{y}{x} - \delta|$ is minimized. Note the two indices i^*, j^* of the optimal solution $S(i^*, j^*)$ can be found if we record some information when the addition operation of two points is performed. By Theorem 8, it can be done in $O(n \log n)$ time, so we have the following theorem.

Theorem 10: The DENSITY FINDING PROBLEM can be solved in optimal $O(n \log n)$ time.

7 Open Problems

It remains open whether the MINKOWSKI SUM SELECTION PROBLEM with a linear objective function and λ constraints can be solved in $O(n \log n)$ time for any fixed $\lambda \geq 2$.

8 Acknowledgment

We thank our advisor Prof. Kun-Mao Chao for valuable comments. Cheng-Wei Luo, Peng-An Chen and Hsiao-Fei Liu were supported in part by NSC grants 95-2221-E-002-126-MY3 and 96-2221-E-002-034 from the National Science Council, Taiwan.

References

- [1] L. Allison. Longest Biased Interval and Longest Non-negative Sum Interval. *Bioinformatics Application Note*, 19(10):1294–1295, 2003.
- [2] M. Ben-Or. Lower Bounds for Algebraic Computation Trees. *STOC*, 80–86, 1983.
- [3] F. Bengtsson and J. Chen. Efficient Algorithms for k Maximum Sums. *ISAAC*, 137–148, 2004.
- [4] M. de Berg, M. van Kreveld, M. Overmars, R. L. Rivest, and Otfried Schwarzkopf. *Computational Geometry: Algorithms and Applications*. Springer, second edition, 2000.
- [5] T. Bernholt, F. Eisenbrand, and T. Hofmeister. A Geometric Framework for Solving Subsequence Problems in Computational Biology Efficiently. *SCG*, 310–318, 2007.
- [6] H. Brönnimann and B. Chazelle. Optimal Slope Selection via Cuttings. *Computational Geometry*, 10(1):23–29, 1998.
- [7] R. Cole, J. S. Salowe, W. L. Steiger, and E. Szemerédi. An Optimal-Time Algorithm for Slope Selection. *SIAM Journal on Computing*, 18(4):792–810, 1989.
- [8] M. B. Dillencourt, D. M. Mount, and N. S. Nethanyahu. A Randomized Algorithm for Slope Selection. *International Journal of Computational Geometry and Applications*, 2(1):1–27, 1992.
- [9] G. N. Frederickson and D. B. Johnson. Generalized Selection and Ranking: Sorted Matrices. *SIAM Journal on Computing*, 13(1):14–30, 1984.
- [10] M. H. Goldwasser, M.-Y. Kao, and H.-I. Lu. Linear-time Algorithms for Computing Maximum-density Sequence Segments with Bioinformatics Applications. *Journal of Computer and System Sciences*, 70(2):128–144, 2005.
- [11] X. Huang. An Algorithm for Identifying Regions of a DNA Sequence that Satisfy a Content Requirement. *Computer Applications in the Biosciences*, 10(3):219–225, 1994.
- [12] M. J. Katz and M. Sharir. Optimal Slope Selection via Expanders. *Information Processing Letters*, 47(3):115–122, 1993.
- [13] D. T. Lee, T.-C. Lin, and H.-I. Lu. Fast Algorithms for the Density Finding Problem. *Algorithmica*, DOI:10.1007/s00453-007-9023-8, 2007.
- [14] T.-C. Lin and D. T. Lee. Efficient Algorithm for the Sum Selection Problem and k Maximum Sums Problem. *ISAAC*, 460–473, 2006.
- [15] T.-C. Lin and D. T. Lee. Randomized Algorithm for the Sum Selection Problem. *Theoretical Computer Science*, 377(1-3):151–156, 2007.

- [16] Y.-L. Lin, T. Jiang, and K.-M. Chao. Efficient Algorithms for Locating the Length-constrained Heaviest Segments with Applications to Biomolecular Sequence Analysis. *Journal of Computer and System Sciences*, 65(3):570–586, 2002.
- [17] D. Lipson, Y. Aumann, A. Ben-Dor, N. Linial, and Z. Yakhini. Efficient Calculation of Interval Scores for DNA Copy Number Data Analysis. *Journal of Computational Biology*, 13(2):215-228, 2006.
- [18] J. Matoušek, D. M. Mount, and N. S. Netanyahu. Efficient Randomized Algorithms for the Repeated Median Line Estimator. *Algorithmica*, 20(2):136–150, 1998.
- [19] L. Wang and Y. Xu. SEGID: Identifying Interesting Segments in (Multiple) Sequence Alignments. *Bioinformatics*, 19(2):297-298, 2003.